

Homework 1

From Data to Intelligent Model

Hands-on AI · SEMFE, NTUA

Deadline: Thursday 09/04 23:59

Overview

In this assignment you will build a complete machine learning pipeline from scratch. You will select a domain, acquire or generate a dataset, preprocess the data, train and compare two models – a classical ML algorithm and a neural network – and evaluate both rigorously.

Important: The best-performing model and preprocessing pipeline you produce here **will be reused in Homework 2 and the Final Project**. Invest time in keeping your code clean, modular, and well-documented. Which model carries forward is determined by your evaluation results in Task 4 – not decided in advance.

The mandatory tasks are **Tasks 1 through 4**. Tasks 5 and 6 are optional and yield **bonus marks**.

Required Project Structure

Submit your work as a **GitHub repository URL**. The following structure is required:

```
hw1/
|-- main.py                # Entry point: runs the full
    pipeline
|-- data/
|   '-- dataset.csv        # Your dataset
|-- models/
|   |-- classical_model.pkl # Saved classical ML model
|   |-- neural_network.*   # Saved neural network (.pt or .
    h5)
|   |-- best_model.pkl     # Copy of the best performing
    model
|   '-- scaler.pkl         # Saved preprocessing scaler
|-- src/
|   |-- preprocessing.py   # All preprocessing logic
|   |-- train_classical.py # Classical ML training logic
|   |-- train_neural.py   # Neural network training logic
|   '-- evaluate.py        # Evaluation and visualization
|-- requirements.txt
'-- README.md
```

Task 1. Domain & Dataset Selection

Choose a domain of interest and obtain a dataset with **at least 8,000 rows and 8 columns**. You have two options:

- **Option A – Real Dataset:** Download a publicly available dataset from sources such as [Kaggle](#), the [UCI ML Repository](#), or [Google Dataset Search](#). Make sure the dataset you choose is **raw and unprocessed** – it should contain missing values, outliers, or categorical variables that require treatment. A dataset that has already been cleaned defeats the purpose of Task 2 and will be penalised accordingly.
- **Option B – Synthetic Dataset:** Generate a realistic synthetic dataset using statistical distributions (e.g. Normal, Poisson, Exponential) that approximate real-world behaviour in your chosen domain.

Regardless of the option chosen, your dataset must contain:

- At least **8,000 rows and 8 columns**
- A mix of **numerical and categorical** features
- At least **one clear target variable** (what you will predict)

Your problem must be either a **classification** task (predicting a discrete category) or a **regression** task (predicting a continuous value). You will train both a classical ML model and a neural network on this same problem, so the target variable must be well-defined and supervised.

Example:

Domain – E-commerce retail transactions.

Features – `customer_age` (int), `product_category` (categorical: Electronics, Clothing, Books), `price` (float, log-normal), `quantity` (int, Poisson), `discount_pct` (categorical: 0, 5, 10, 15, 20), `region` (categorical: North, South, East, West), `is_member` (binary), `days_since_last_purchase` (int, exponential).

Target variable – `returned` (binary: 0 = not returned, 1 = returned). Problem type: classification. Total rows: 10,000.

Task 2. Preprocessing Pipeline

Build a clean, reusable preprocessing pipeline. All logic should live in `src/preprocessing.py` as functions or a class, so it can be imported and reapplied in future assignments without modification.

Order matters – split first, preprocess second. A common and serious mistake is to compute imputation statistics (e.g. the median for missing value treatment) or outlier boundaries (e.g. IQR bounds) on the full dataset before splitting. This is **data leakage**: information from the validation and test sets silently influences your preprocessing. The correct workflow is: **(1) split the data, then (2) compute all statistics on the training set only, then (3) apply them to all three sets.** The subsections below follow this order exactly.

2.1. Train / Validation / Test Split

The very first step in your pipeline – before any imputation, outlier treatment, encoding, or scaling – is to split your dataset into three non-overlapping partitions.

- **Training set (80%):** used to fit all preprocessing steps and both models.
- **Validation set (10%):** used during model training to monitor overfitting and guide early stopping. It also serves as a final confirmation of model selection before the test set is ever touched.
- **Test set (10%):** used once at the very end to report final performance. Never use it during training or tuning.

The standard approach is to perform this in two sequential splits: first hold out 10% as the test set, then from the remaining 90% hold out approximately 11%, which yields the 80/10/10 proportion overall.

For **classification problems**, use stratified splitting to preserve the class ratio across all three partitions. For **time series data**, do not shuffle – split chronologically instead.

Example:

The e-commerce dataset (10,000 rows) is split 80/10/10. Because **returned** is imbalanced (85% not returned, 15% returned), **stratify=y** is used in both split steps to preserve the class ratio. Final shapes: Train (8,000 rows), Validation (1,000 rows), Test (1,000 rows). All subsequent preprocessing statistics are computed exclusively on the 8,000 training rows.

2.2. Missing Value Treatment

Detect and handle missing values in each column. All imputation statistics (medians, modes, fill values) must be **computed on the training set only** and then applied identically to the validation and test sets.

Common strategies:

- **Numerical columns:** impute with mean or median (prefer median for skewed distributions).
- **Categorical columns:** impute with the mode or a dedicated "Unknown" category.
- **Target column:** drop rows where the target is missing.

Example:

In the training set, **price** has 3% missing values. Because **price** is right-skewed, the median of `X_train["price"]` is computed and used for imputation. This same median value is then used to fill missing entries in `X_val` and `X_test` – their own medians are never computed separately. **product_category** missing values are filled with "Unknown" across all three sets.

2.3. Outlier Detection and Treatment

Apply at least one method to detect and handle outliers in numerical features. All boundary values must be **computed on the training set only** and then applied to all three

sets.

Two standard approaches:

- **IQR method:** flag values below $Q_1 - 1.5 \cdot \text{IQR}$ or above $Q_3 + 1.5 \cdot \text{IQR}$.
- **Z-score method:** flag values with $|z| > 3$.

Once detected, you may either **remove** the outlying rows or **cap** (Winsorize) the values at the boundary – justify your choice.

Example:

The IQR upper bound for `price` is calculated on `X_train` as \$450. Values above \$450 in `X_train`, `X_val`, and `X_test` are all capped at \$450 (Winsorizing). The \$450 bound is never recalculated on the validation or test sets.

2.4. Encoding Categorical Variables

Convert categorical columns into numerical form. Choose the encoding method based on the nature of each variable:

- **One-Hot Encoding:** for nominal categories with no inherent order (e.g. `region`, `product_category`).
- **Label Encoding:** for binary columns or ordinal categories (e.g. `is_member`).
- **Target Encoding:** for high-cardinality categorical columns where one-hot would produce too many features.

Example:

`product_category` (5 unique values) is one-hot encoded, producing 4 binary columns (`drop_first=True` to avoid multicollinearity). `region` (4 values) is also one-hot encoded. `is_member` is already binary (0/1) and requires no transformation.

2.5. Feature Engineering

Create at least **2 new features** derived from existing columns. New features should encode domain knowledge that the model cannot easily learn from raw inputs. These features are **added to the dataset** alongside the original columns – they do not replace them. Your dataset will therefore have more than 8 columns after this step, which is expected.

Example:

Feature 1 – `revenue = price × quantity × (1 – discount_pct/100)`: captures the actual monetary value of the transaction, more predictive of return behaviour than price or quantity alone.

Feature 2 – `engagement_score = 1/(days_since_last_purchase+1)`: captures customer recency in a bounded, non-linear form. A customer who purchased yesterday scores near 1; one who last purchased 365 days ago scores near 0.

Both features are added as new columns. The original `price`, `quantity`, `discount_pct`, and `days_since_last_purchase` columns remain in the dataset.

2.6. Feature Scaling

Apply scaling to numerical features so that no single feature dominates due to its range. The scaler must be **fitted on the training set only**, then applied to all three sets. Save the fitted scaler object – you will need it in Homework 2.

- **StandardScaler:** transforms to zero mean and unit variance. Preferred for linear models and neural networks.
- **MinMaxScaler:** scales to a $[0, 1]$ range. Preferred when the distribution is bounded or uniform.
- **RobustScaler:** uses median and IQR instead of mean and std. Preferred when outliers remain after treatment.

Example:

StandardScaler is fitted on `X_train` and then used to transform `customer_age`, `price`, `quantity`, `days_since_last_purchase`, `revenue`, and `engagement_score` across all three splits. The fitted scaler is saved as `models/scaler.pkl`.

2.7. Exploratory PCA (Dimensionality Insight)

After scaling, run a **Principal Component Analysis (PCA)** on the training feature set as a mandatory exploratory step. The goal here is *not* to reduce dimensions for modelling – it is to understand which features carry the most information and how much variance each principal component explains.

Why PCA must come after scaling: PCA directions are determined purely by variance. On unscaled data, features with large numerical ranges (e.g. `revenue` in the thousands) will dominate all principal components regardless of their true informativeness. Always run PCA on scaled data.

Specifically, you should:

- Plot the **explained variance ratio** per component (scree plot) to see how many components are needed to capture, say, 90% of the total variance.
- Inspect the **PCA loadings** (component weights) to identify which original features contribute most to the first two or three principal components.
- Produce a **2D scatter plot** of your data projected onto the first two principal components, coloured by target class.

The insights from this step should be discussed in your README.

Example:

PCA is run on the scaled `X_train` (10 features post-engineering). The scree plot shows that the first 3 components explain 74% of total variance and the first 5 explain 91%. Inspecting the loadings reveals that `revenue`, `price`, and `engagement_score` dominate the first principal component, confirming that transaction value and customer recency are the most informative axes of variation. The 2D scatter plot shows a rough separation between returned and non-returned transactions along PC1.

Task 3. Model Training

You will train **two models** on the same preprocessed dataset: a classical ML algorithm and a neural network. Training both on identical data under identical evaluation conditions allows you to make a fair, evidence-based comparison in Task 4. The model that performs better on the validation set will be designated `best_model.pkl` and carried forward into Homework 2 and the Final Project.

Do not assume the neural network will win. On tabular datasets of this size, classical ML algorithms – particularly tree-based models like Random Forest and XGBoost – consistently match or outperform simple neural networks. This is a well-established empirical finding. Your comparison in Task 4 may confirm this, and that is a valid and valuable result.

3.1. Classical ML Model

Train at least one classical machine learning model appropriate for your problem type:

Problem Type	Description	Suggested Algorithms
Classification	Predict a discrete category	Random Forest, XGBoost, Logistic Regression, SVM
Regression	Predict a continuous value	Ridge, Lasso, Polynomial, SVR, Gradient Boosting

Note on Regression: For regression problems, use a regularised model (e.g. Ridge, Lasso) rather than plain Linear Regression. Regularisation controls model complexity, handles correlated features more robustly, and is standard practice on real-world data. Plain Linear Regression is a special case with no regularisation and will almost always be outperformed.

Train on `X_train` / `y_train` only. For algorithms that support it (e.g. XGBoost, LightGBM), pass `X_val` / `y_val` as an `eval_set` to monitor validation performance during training and enable early stopping. After training, inspect **feature importances** (tree-based models) or **model coefficients** (linear models). Save the trained model as `models/classical_model.pkl`.

Example:

Problem type – Binary classification.

Model chosen – `RandomForestClassifier` with `n_estimators=100`, `max_depth=10`, and `class_weight='balanced'` to account for class imbalance.

Validation performance – ROC-AUC on `X_val`: 0.89.

Feature importance – Top features: `revenue`, `days_since_last_purchase`, `discount_pct`.

3.2. Neural Network

Train a feedforward neural network on the same preprocessed data. Keep the architecture simple and focused – the goal is a fair comparison, not an exhaustively tuned deep learning system.

Architecture requirements:

- At least **2 hidden layers**
- Use **ReLU** as your default activation function in hidden layers. You are encouraged to experiment with at least one alternative (e.g. Leaky ReLU, ELU, Tanh) and report in your README how the choice affected training behaviour and validation performance
- Appropriate output activation for your problem: **Sigmoid** for binary classification, **Softmax** for multi-class, **linear** (no activation) for regression
- **Dropout** (e.g. 0.2–0.3) on at least one hidden layer for regularisation

Training requirements:

- Use the **validation set** (X_{val} / y_{val}) to monitor validation loss at each epoch
- Implement **early stopping**: halt training when validation loss has not improved for a set number of epochs (patience), and restore the weights from the best epoch
- Plot the **training and validation loss curves** across epochs

You may use either **PyTorch** or **Keras/TensorFlow** although **PyTorch** is recommended. Save the trained model as `models/neural_network.pt` (PyTorch) or `models/neural_network.h5` (Keras).

Example:

Architecture – `Input(10) → Dense(64, ReLU) → Dropout(0.3) → Dense(32, ReLU) → Dense(1, Sigmoid)`.

Training – Adam optimizer, binary cross-entropy loss, batch size 64, early stopping with `patience=10` monitoring `val_loss`.

Validation performance – Best ROC-AUC on X_{val} : 0.85 (reached at epoch 34, training stopped at epoch 44).

Observation – Validation loss plateaus around epoch 25 while training loss continues to decrease, indicating mild overfitting that dropout partially mitigates.

Task 4. Evaluation & Model Comparison

Evaluate **both models** on the **test set only**. Use the same metrics for both so the comparison is fair. Then designate the better-performing model as your best model.

4.1. Metrics by Problem Type

Classification – Report: Accuracy, Precision, Recall, F1-score, ROC-AUC. Produce a **Confusion Matrix** for each model.

Note on Accuracy: For imbalanced datasets, accuracy alone is misleading. A model that always predicts “not returned” achieves 85% accuracy while being completely

useless. Always report Precision, Recall, and F1-score alongside accuracy.

Regression – Report: MAE, RMSE, R^2 . Produce a **Residual Plot** for each model.

4.2. Side-by-Side Comparison

Present your results in a summary table comparing both models across all reported metrics. Do not just report numbers – discuss the differences. Consider:

- Which model performs better and by how much?
- Does the neural network’s loss curve suggest it had enough data to learn well, or did it struggle to converge?
- Does the classical model’s feature importance align with what the PCA loadings suggested in Task 2?
- Is the performance difference large enough to justify the added complexity of the neural network?

Example:

Metric	Random Forest	Neural Network
Accuracy	0.87	0.84
Precision	0.71	0.67
Recall	0.68	0.61
F1-score	0.69	0.64
ROC-AUC	0.91	0.87

The Random Forest outperforms the neural network on all metrics. The neural network’s loss curves show that validation loss stopped improving after epoch 34, suggesting the model extracted most available signal from 8,000 rows but could not generalise further. The Random Forest’s top features (revenue, recency, discount) are consistent with the dominant PCA loadings. The Random Forest is designated the best model.

4.3. Designating the Best Model

After completing the comparison, copy the better-performing model to `models/best_model.pkl` (or `.pt` / `.h5` if the neural network wins). This file is what Homework 2 will load. Document your choice clearly in the README.

Note for future assignments: The best model you designate here will be loaded as a prediction tool inside your AI agent in Homework 2, and retrained on a larger benchmark dataset in the Final Project. Choose honestly based on your test set results.

Task 5. FastAPI Endpoint (Bonus)

Expose your best model as a REST API using **FastAPI**. Create a `/predict` POST endpoint that accepts input features as a JSON payload, applies the saved preprocessing pipeline, and returns the model’s prediction.

Use Pydantic to define the input schema for automatic validation and documentation. Run the application with uvicorn and verify it through the auto-generated Swagger UI at <http://127.0.0.1:8000/docs>.

Example:

Input payload – {"customer_age": 34, "price": 89.99, "quantity": 2, "discount_pct": 10, "is_member": 1, "region": "North", "product_category": "Electronics", "days_since_last_purchase": 15}
Response – {"prediction": 0, "label": "Not Returned", "probability": 0.12}

Task 6. Hyperparameter Tuning (Bonus)

Improve your classical ML model by systematically searching for the best hyperparameter combination. Use `RandomizedSearchCV` with K-Fold cross-validation on `X_train`. After the search is complete, evaluate the best found model on `X_val` to confirm the improvement before finally reporting on `X_test`. Report the best parameter values found and compare performance against the baseline.

Example:

Search space – `n_estimators`: [50, 100, 200], `max_depth`: [5, 10, 15, None], `min_samples_split`: [2, 5, 10].
Result – Best params: `n_estimators=200`, `max_depth=15`, `min_samples_split=5`.
Validation confirmation – ROC-AUC on `X_val`: 0.89 (baseline) → 0.92 (tuned).
Test set improvement – ROC-AUC: 0.91 → 0.93.

Deliverables Checklist

File	Description	Required
main.py	Runs the full pipeline end-to-end	✓
data/dataset.csv	Your dataset	✓
models/classical_model.pkl	Saved classical ML model	✓
models/neural_network.*	Saved neural network	✓
models/best_model.pkl	Copy of the best model	✓
models/scaler.pkl	Saved preprocessing scaler	✓
src/preprocessing.py	Preprocessing functions / class	✓
src/train_classical.py	Classical ML training logic	✓
src/train_neural.py	Neural network training logic	✓
src/evaluate.py	Evaluation and plots	✓
requirements.txt	All Python dependencies	✓
README.md	Documentation (see below)	✓
src/api.py	FastAPI /predict endpoint	★ Bonus

README Requirements

Your `README.md` must contain the following sections:

1. **Problem Description** – What domain did you choose? Is it a classification or regression problem? What is the target variable and why is it a useful prediction task?
2. **Dataset Description** – Source (URL or generation method), number of rows and columns, description of each feature, class or value distribution of the target variable.
3. **Preprocessing Approach** – Which strategies were applied for missing values, outliers, encoding, and scaling, and why. Confirm that all statistics were derived from the training set only.
4. **Feature Engineering** – What new features were created and what is their intuition?
5. **PCA Insights** – What does the scree plot reveal? Which features dominated the first principal components? What does the 2D projection show?
6. **Model Comparison** – Present results for both models side by side. Which model won and why? Does the outcome surprise you given your dataset size and domain?
7. **Best Model Designation** – Which model is `best_model.pkl` and what justifies that choice?
8. **Installation & Execution** – Step-by-step commands to clone the repo, install dependencies, and run the pipeline.

Key Reminders

Split First, Preprocess Second: Always split your data before computing any preprocessing statistics. Medians, IQR bounds, and scaler parameters must all be derived from the training set only and then applied to validation and test sets.

Save Everything: Your `best_model.pkl` and `scaler.pkl` will be loaded directly in Homework 2. If they are missing or were not saved correctly, you will need to retrain from scratch.

Reproducibility: Set `random_state=42` everywhere seeds are accepted (`train/test/split`, model constructors, search objects) so your results can be reproduced exactly.

Good luck! The foundation you build here will carry you through all three homeworks.